Configmap and secret file

```
Editor   Tab 1   +

controlplane:~$ ls
filesystem
controlplane:~$ git clone https://github.com/dharaniobulesu/DevOps-Notes.git
Cloning into 'DevOps-Notes'...
remote: Enumerating objects: 29, done.
remote: Counting objects: 100% (29/29), done.
remote: Compressing objects: 100% (27/27), done.
remote: Total 29 (delta 7), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (29/29), 17.04 MiB | 7.70 MiB/s, done.
Resolving deltas: 100% (7/7), done.
controlplane:~$ ls
DevOps-Notes  filesystem
controlplane:~$ rm -rf DevOps-Notes
controlplane:~$ ls
filesystem
controlplane:~$ git clone https://github.com/dharaniobulesu/fss-Retail-App_kubernetes.git
Cloning into 'fss-Retail-App_kubernetes'...
remote: Enumerating objects: 2424, done.
remote: Counting objects: 100% (145/145), done.
remote: Compressing objects: 100% (103/103), done.
remote: Total 2424 (delta 82), reused 46 (delta 38), pack-reused 2279 (from 2)
Receiving objects: 100% (2424/2424), 9.40 MiB | 9.48 MiB/s, done.
Resolving deltas: 100% (550/550), done.
controlplane:~$ ls
filesystem  fss-Retail-App_kubernetes
controlplane:~$ cd fss-Retail-App_kubernetes/
controlplane:~/fss-Retail-App_kubernetes$ ls
Dockerfile  Public     docker-compose.yaml  node_modules       package.json  test-volumes.yaml
```

```
controlplane:~/fss-Retail-App_kubernetes$ cd k8s-manifests/
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ l
mongodb-deployment.yml  mongodb-service.yml  usernode-js-service.yml  userprofile-deployment.yml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ sls
Command 'sls' not found, but there are 21 similar ones.
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ ls
mongodb-deployment.yml  mongodb-service.yml  usernode-js-service.yml  userprofile-deployment.yml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ vi configmap.yaml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ ls
configmap.yaml  mongodb-deployment.yml  mongodb-service.yml  usernode-js-service.yml  userprofile-deployment.yml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ cat configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: retail-app-config
  namespace: dharani-ns
data:
  MONGODB_URI: "mongodb://mongodb:27017/myDatabase"
  SESSION_SECRET: "1234"
  PORT: "3130"
  MONGO_INITDB_DATABASE: "myDatabase"
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ kubectl create ns dharani-ns
namespace/dharani-ns created
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ kubectl apply -f configmap.yaml
configmap/retail-app-config created
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ mv user
usernode-js-service.yml     userprofile-deployment.yml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ mv userprofile-deployment.yml retail-app-deployment.yaml
```

```
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ kubectl apply -f con
configmap/retail-app-config created
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ mv user
usernode-js-service.yml     userprofile-deployment.yml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ mv userprofile-deplo
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ vi retail-app-deploy
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ kubectl apply -f ret
deployment.apps/retail-app-deployment created
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ mv usernode-js-servi
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ vi retail-app-servic
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ ls
configmap.yaml  mongodb-deployment.yml  mongodb-service.yml  retail-app-depl
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ kubectl apply -f ret
```

```
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ kubectl apply -f configmap.yaml
configmap/retail-app-config created
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ mv user
usernode-js-service.yml     userprofile-deployment.yml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ mv userprofile-deployment.yml retail-app-deployment.yaml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ vi retail-app-deployment.yaml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ kubectl apply -f retail-app-deployment.yaml
deployment.apps/retail-app-deployment created
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ mv usernode-js-service.yml retail-app-service.yaml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ vi retail-app-service.yaml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ ls
configmap.yaml  mongodb-deployment.yml  mongodb-service.yml  retail-app-deployment.yaml  retail-app-service.yaml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ kubectl apply -f retail-app-service.yaml
service/retail-app-service created
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ ls
configmap.yaml  mongodb-deployment.yml  mongodb-service.yml  retail-app-deployment.yaml  retail-app-service.yaml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ vi mongodb-deployment.yml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ kubectl apply -f mongodb-deployment.yml
deployment.apps/retail-mongodb created
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ vi mongodb-service.yml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ kubectl apply -f mongodb-service.yml
service/mongodb created
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ kubectl get all -n dharani-ns
```

```
NAME                                    READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/retail-app-deployment   3/4     2            3           16m
deployment.apps/retail-mongodb          1/1     1            1           10m

NAME                                               DESIRED   CURRENT   READY   AGE
replicaset.apps/retail-app-deployment-549f776b58   3         3         3       16m
replicaset.apps/retail-app-deployment-64c485cbd7   2         2         0       54s
replicaset.apps/retail-mongodb-58ffcb7cf9          1         1         1       10m
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ vi secret.yaml
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ kubectl apply -f secret.yaml
secret/retail-app-secret created
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ kubectl get all -n dharani-ns
NAME                                        READY   STATUS    RESTARTS   AGE
pod/retail-app-deployment-64c485cbd7-42zmv  1/1     Running   0          69s
pod/retail-app-deployment-64c485cbd7-7jjxd  1/1     Running   0          3m58s
pod/retail-app-deployment-64c485cbd7-kh9ld  1/1     Running   0          67s
pod/retail-app-deployment-64c485cbd7-nzctx  1/1     Running   0          3m58s
pod/retail-mongodb-58ffcb7cf9-x76p7         1/1     Running   0          13m

NAME                        TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/mongodb             ClusterIP      10.101.1.108    <none>        27017/TCP        11m
service/retail-app-service  LoadBalancer   10.101.46.170   <pending>     3130:31379/TCP   16m

NAME                                    READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/retail-app-deployment   4/4     4            4           19m
deployment.apps/retail-mongodb          1/1     1            1           13m
```

```
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$ history
    1  exit
    2  halt
    3  ls
    4  git clone https://github.com/dharaniobulesu/DevOps-Notes.git
    5  ls
    6  rm -rf DevOps-Notes
    7  ls
    8  git clone https://github.com/dharaniobulesu/fss-Retail-App_kubernetes.git
    9  ls
   10  cd fss-Retail-App_kubernetes/
   11  ls
   12  cd k8s-manifests/
   13  l
   14  sls
   15  ls
   16  vi configmap.yaml
   17  ls
   18  cat configmap.yaml
   19  kubectl create ns dharani-ns
   20  kubectl apply -f configmap.yaml
   21  mv userprofile-deployment.yml retail-app-deployment.yaml
   22  vi retail-app-deployment.yaml
   23  kubectl apply -f retail-app-deployment.yaml
   24  mv usernode-js-service.yml retail-app-service.yaml
   25  vi retail-app-service.yaml
   26  ls
   27  kubectl apply -f retail-app-service.yaml
```

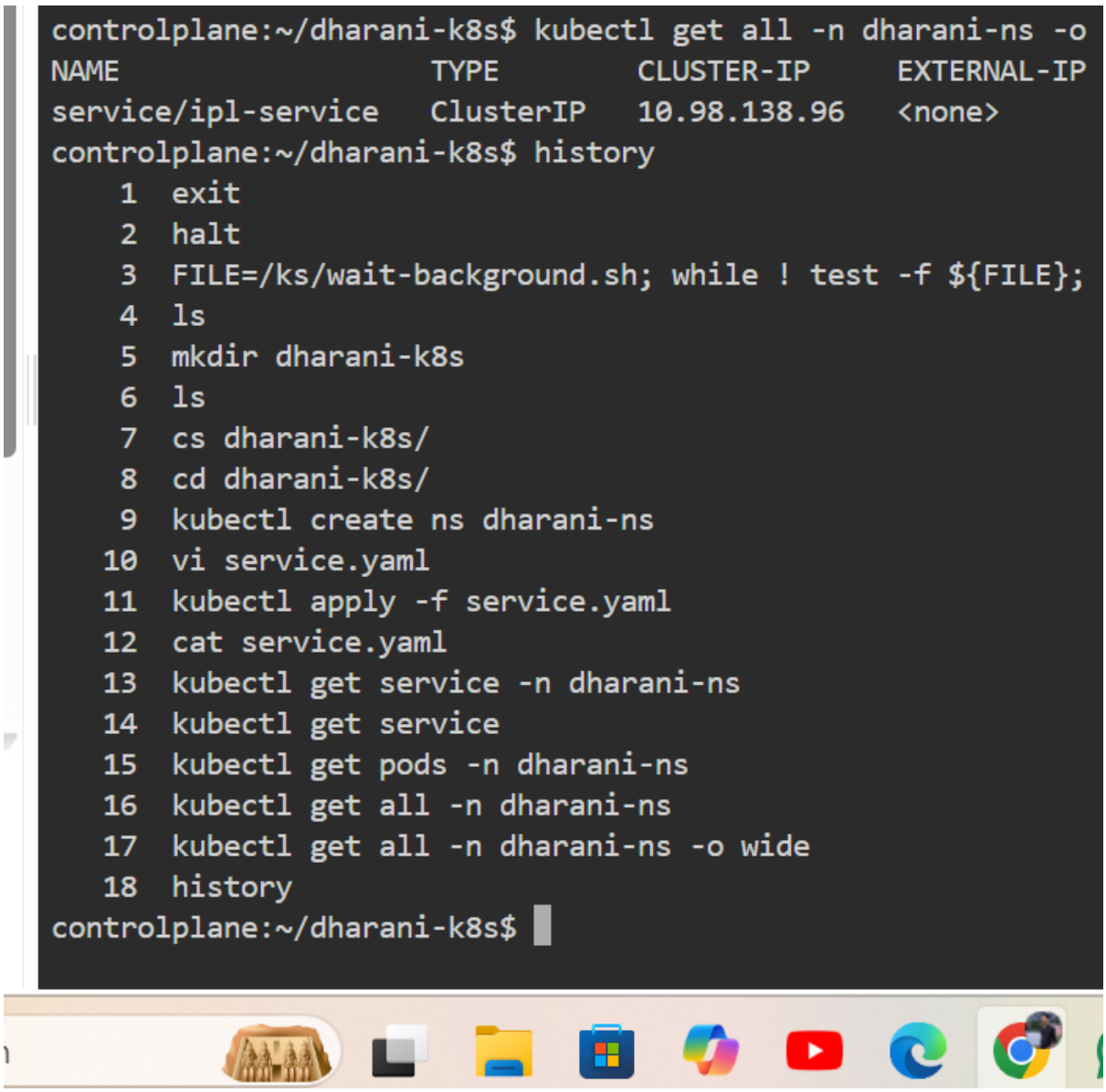
```
   22  vi retail-app-deployment.yaml
   23  kubectl apply -f retail-app-deployment.yaml
   24  mv usernode-js-service.yml retail-app-service.yaml
   25  vi retail-app-service.yaml
   26  ls
   27  kubectl apply -f retail-app-service.yaml
   28  ls
   29  vi mongodb-deployment.yml
   30  kubectl apply -f mongodb-deployment.yml
   31  vi mongodb-service.yml
   32  kubectl apply -f mongodb-service.yml
```

```
   22  vi retail-app-deployment.yaml
   23  kubectl apply -f retail-app-deployment.yaml
   24  mv usernode-js-service.yml retail-app-service.yaml
   25  vi retail-app-service.yaml
   26  ls
   27  kubectl apply -f retail-app-service.yaml
   28  ls
   29  vi mongodb-deployment.yml
   30  kubectl apply -f mongodb-deployment.yml
   31  vi mongodb-service.yml
   32  kubectl apply -f mongodb-service.yml
   33  kubectl get all -n dharani-ns
   34  kubectl get service -n dharani-ns
   35  ls
   36  vi retail-app-deployment.yaml
   37  kubectl apply -f retail-app-deployment.yaml
   38  kubectl get all -n dharani-ns
   39  vi secret.yaml
   40  kubectl apply -f secret.yaml
   41  kubectl get all -n dharani-ns
   42  history
controlplane:~/fss-Retail-App_kubernetes/k8s-manifests$
```

```
controlplane:~$ ls
filesystem
controlplane:~$ mkdir dharani-k8s
controlplane:~$ ls
dharani-k8s  filesystem
controlplane:~$ cs dharani-k8s/
Command 'cs' not found, but can be installed with:
apt install csound
controlplane:~$ cd dharani-k8s/
controlplane:~/dharani-k8s$ kubectl create ns dharani-ns
namespace/dharani-ns created
controlplane:~/dharani-k8s$ vi service.yaml
controlplane:~/dharani-k8s$ kubectl apply -f service.yaml
service/ipl-service created
controlplane:~/dharani-k8s$ cat service.yaml
apiVersion: v1
kind: Service
metadata:
  name: ipl-service
  namespace: dharani-ns
spec:
  selector:
    app: ipl
  ports:
    - protocol: TCP
      port: 80         # host port or service port, it can be ar
iner port
```

```
  namespace: dharani-ns
spec:
  selector:
    app: ipl
  ports:
    - protocol: TCP
      port: 80         # host port or service port, it can be any port number, it is not necessary to be s
iner port
      targetPort: 3000 # container port, it should be same as container port defined in deployment.yaml
  type: ClusterIP
controlplane:~/dharani-k8s$ kubectl get service -n dharani-ns
NAME          TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
ipl-service   ClusterIP   10.98.138.96   <none>        80/TCP    79s
controlplane:~/dharani-k8s$ kubectl get service
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   21d
controlplane:~/dharani-k8s$ kubectl get pods -n dharani-ns
No resources found in dharani-ns namespace.
controlplane:~/dharani-k8s$ kubectl get all -n dharani-ns
NAME                  TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
service/ipl-service   ClusterIP   10.98.138.96   <none>        80/TCP    2m14s
controlplane:~/dharani-k8s$ kubectl get all -n dharani-ns -o wide
NAME                  TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE     SELECTOR
service/ipl-service   ClusterIP   10.98.138.96   <none>        80/TCP    2m36s   app=ipl
controlplane:~/dharani-k8s$
```

Service:


controlplane:~/dharani-k8s$ kubectl get all -n dharani-ns -o
NAME TYPE CLUSTER-IP EXTERNAL-IP
service/ipl-service ClusterIP 10.98.138.96 <none>
controlplane:~/dharani-k8s$ history
1 exit
2 halt
3 FILE=/ks/wait-background.sh; while ! test -f ${FILE};
4 ls
5 mkdir dharani-k8s
6 ls
7 cs dharani-k8s/
8 cd dharani-k8s/
9 kubectl create ns dharani-ns
10 vi service.yaml
11 kubectl apply -f service.yaml
12 cat service.yaml
13 kubectl get service -n dharani-ns
14 kubectl get service
15 kubectl get pods -n dharani-ns
16 kubectl get all -n dharani-ns
17 kubectl get all -n dharani-ns -o wide
18 history
controlplane:~/dharani-k8s$

```
controlplane:~$ mkdir dharani-k8s
controlplane:~$ ls
dharani-k8s  filesystem
controlplane:~$ cd dharani-k8s/
controlplane:~/dharani-k8s$ kubectl create ns dharani-ns
namespace/dharani-ns created
controlplane:~/dharani-k8s$ vi netflix-service.yaml
controlplane:~/dharani-k8s$ vi netflix-deployment.yaml
controlplane:~/dharani-k8s$ vi netflix-ingress.yaml
controlplane:~/dharani-k8s$ kubectl apply -f netflix-deployment.yaml
deployment.apps/netflix-deployment created
controlplane:~/dharani-k8s$ kubectl apply -f netflix-service.yaml
service/netflix-service created
controlplane:~/dharani-k8s$ kubectl apply -f netflix-ingress.yaml
ingress.networking.k8s.io/ipl-ingress created
controlplane:~/dharani-k8s$ kubectl get deployment -n dharani-ns
NAME                 READY   UP-TO-DATE   AVAILABLE   AGE
netflix-deployment   5/5     5            5           59s
controlplane:~/dharani-k8s$ kubectl get service -n dharani-ns
NAME              TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)   AGE
netflix-service   ClusterIP   10.111.152.210   <none>        80/TCP    54s
controlplane:~/dharani-k8s$ kubectl get ingress -n dharani-ns
NAME          CLASS   HOSTS   ADDRESS   PORTS   AGE
ipl-ingress   nginx   *                 80      53s
controlplane:~/dharani-k8s$ kubectl get all -n dharani-ns
NAME                                     READY   STATUS    RESTARTS   AGE
pod/netflix-deployment-569df6b59-6sp5c   1/1     Running   0          101s
pod/netflix-deployment-569df6b59-7nklp   1/1     Running   0          101s
```

```
NAME                      TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)   AGE
service/netflix-service   ClusterIP   10.111.152.210   <none>        80/TCP    84s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/netflix-deployment   5/5     5            5           101s

NAME                                           DESIRED   CURRENT   READY   AGE
replicaset.apps/netflix-deployment-569df6b59   5         5         5       101s
controlplane:~/dharani-k8s$ vi deployment.yaml
controlplane:~/dharani-k8s$ vi service.yaml
controlplane:~/dharani-k8s$ vi ingress.yaml
controlplane:~/dharani-k8s$ kubectl apply -f deployment.yaml
deployment.apps/ipl-deployment created
controlplane:~/dharani-k8s$ kubectl apply -f service.yaml
service/netflix-service unchanged
controlplane:~/dharani-k8s$ vi service.yaml
controlplane:~/dharani-k8s$ kubectl apply -f service.yaml
service/ipl-service created
controlplane:~/dharani-k8s$ kubectl apply -f ingress.yaml
ingress.networking.k8s.io/ipl-ingress configured
controlplane:~/dharani-k8s$ kubectl get deployment -n dharani-ns
NAME                 READY   UP-TO-DATE   AVAILABLE   AGE
ipl-deployment       5/5     5            5           2m23s
netflix-deployment   5/5     5            5           7m42s
controlplane:~/dharani-k8s$ kubectl get service -n dharani-ns
NAME              TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
ipl-service       LoadBalancer   10.97.131.0      <pending>     80:31567/TCP   60s
netflix-service   ClusterIP      10.111.152.210   <none>        80/TCP         7m40s
```

```
 4  ls
 5  mkdir dharani-k8s
 6  ls
 7  cd dharani-k8s/
 8  kubectl create ns dharani-ns
 9  vi netflix-service.yaml
10  vi netflix-deployment.yaml
11  vi netflix-ingress.yaml
12  kubectl apply -f netflix-deployment.yaml
13  kubectl apply -f netflix-service.yaml
14  kubectl apply -f netflix-ingress.yaml
15  kubectl get deployment -n dharani-ns
16  kubectl get service -n dharani-ns
17  kubectl get ingress -n dharani-ns
18  kubectl get all -n dharani-ns
19  vi deployment.yaml
20  vi service.yaml
21  vi ingress.yaml
22  kubectl apply -f deployment.yaml
23  kubectl apply -f service.yaml
24  vi service.yaml
25  kubectl apply -f service.yaml
26  kubectl apply -f ingress.yaml
27  kubectl get deployment -n dharani-ns
28  kubectl get service -n dharani-ns
29  kubectl get ingress -n dharani-ns
30  history
```